

```
// ===== WEATHER GATE v1.5-FSK (FULL ADAPTED) =====

#define MQTT_MAX_PACKET_SIZE 1024
#include <ETH.h>
#include <WiFi.h>
#include <Wire.h>
#include <RTCLib.h>
#include <SPI.h>
#include <LittleFS.h>
#include <ArduinoJson.h>
#include <WebServer.h>
#include <PubSubClient.h>
#include <Preferences.h>
#include <HTTPClient.h>
#include <HTTPUpdate.h>
#include <rtl_433_ESP.h>
#include <RadioLib.h>

// Сообщаем об объекте внутри библиотеки rtl_433_ESP
extern CC1101 radio;

// --- ПИНЫ ---
#define I2C_SDA 32
#define I2C_SCL 33
#define LED_RF_RX 25
#define LED_MQTT_TX 26
#define CC1101_CS 2
#define CC1101_GDO0 4
#define CC1101_SCK 12
#define CC1101_MISO 15
#define CC1101_MOSI 14

// --- ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ (Ваша логика v1.5) ---
float current_freq = 915.00;
bool verbose_mode = false;
String hostname = "weather_gate";
// ... (остальные переменные: temperature, humidity, mqtt_ip и т.д. сохраняются из вашего кода) ...

WebServer server(80);
Preferences pref;
WiFiClient ethClient;
PubSubClient mqtt(ethClient);
rtl_433_ESP rf;
RTC_DS3231 rtc;
```

```
// ===== ЛОГИКА РАДИО (НОВАЯ) =====
```

```
void rtl_433_Callback(char* message) {  
    digitalWrite(LED_RF_RX, HIGH);  
    // Ваша логика парсинга JSON и отправки в MQTT  
    Serial.printf("[RF] Received: %s\n", message);  
    digitalWrite(LED_RF_RX, LOW);  
}
```

```
bool initCC1101() {  
    Serial.println("[CC1101] Custom init for WS1080...");  
    // 1. Указываем пины SPI явно перед инициализацией  
    SPI.begin(CC1101_SCK, CC1101_MISO, CC1101_MOSI, CC1101_CS);
```

```
    // 2. Настройка rtl_433_ESP  
    rf.ookModulation = false; // Обязательно FSK  
    // Инициализация. Если вернет не 0, значит чип не найден  
    int state = rf.initReceiver(CC1101_GDO0, current_freq);  
    if (state != RADIOLIB_ERR_NONE) {  
        Serial.printf("[CC1101] Error: Chip not found (Code: %d)\n", state);  
        return false;  
    }
```

```
    // 3. Прямая настройка параметров FSK через RadioLib  
    radio.setRxBandwidth(325.0);  
    radio.setFrequencyDeviation(47.6);  
    radio.setBitRate(17.241);
```

```
    rf.rtlVerbose = verbose_mode ? 1 : 0;  
    Serial.println("[CC1101] Receiver initialized successfully");  
    return true;  
}
```

```
// ===== ВЕБ-ИНТЕРФЕЙС (ВАШ ОРИГИНАЛЬНЫЙ) =====
```

```
void setupWebHandlers() {  
    // Возвращаем все ваши пути из оригинального скетча  
    server.on("/", HTTP_GET, []() {  
        if (!LittleFS.exists("/index.html")) {  
            server.send(404, "text/plain", "FS Error: index.html not found");  
            return;  
        }  
        File f = LittleFS.open("/index.html", "r");  
        server.streamFile(f, "text/html");  
    });
```

```
f.close();  
});
```

```
server.on("/api/status", HTTP_GET, []() {  
  JsonDocument doc;  
  doc["freq"] = current_freq;  
  doc["rssi"] = radio.getRSSI();  
  doc["uptime"] = millis() / 1000;  
  // ... (добавьте остальные поля статуса из вашего v1.5) ...  
  String out;  
  serializeJson(doc, out);  
  server.send(200, "application/json", out);  
});
```

```
server.on("/api/set_freq", HTTP_POST, []() {  
  if (server.hasArg("f")) {  
    current_freq = server.arg("f").toFloat();  
    radio.setFrequency(current_freq);  
    // Сохранение в Preferences  
    pref.begin("weather", false);  
    pref.putFloat("freq", current_freq);  
    pref.end();  
    server.send(200, "text/plain", "OK");  
  }  
});
```

```
server.on("/api/set_verbose", HTTP_POST, []() {  
  verbose_mode = (server.arg("v") == "1");  
  rf.rtlVerbose = verbose_mode ? 1 : 0;  
  server.send(200, "text/plain", "OK");  
});  
// Добавьте сюда остальные ваши server.on из v1.5 (update, restart, settings)  
}
```

```
// ===== ГЛАВНЫЕ ФУНКЦИИ =====
```

```
void setup() {  
  Serial.begin(115200);  
  // 1. Пины индикации  
  pinMode(LED_RF_RX, OUTPUT);  
  pinMode(LED_MQTT_TX, OUTPUT);  
  
  // 2. Файловая система (нужна для работы Web-интерфейса)  
  if (!LittleFS.begin(true)) Serial.println("LittleFS Mount Failed");  
}
```

```
// 3. Загрузка настроек
pref.begin("weather", true);
current_freq = pref.getFloat("freq", 915.00);
pref.end();

// 4. Инициализация радио (До Ethernet, чтобы избежать конфликтов)
initCC1101();
rf.setCallback(rtl_433_Callback);
rf.enableReceiver();

// 5. Сеть (Ваш конфиг WT32-ETH01)
ETH.begin(ETH_PHY_LAN8720, 1, 23, 18, 16, ETH_CLOCK_GPIO17_OUT);
// 6. Настройка путей сервера
setupWebHandlers();
server.begin();
Serial.println("[SYS] System initialized and running");
}

void loop() {
// Даем время Веб-серверу
server.handleClient();
// Даем время Радио-декодеру
rf.loop();
// Обработка MQTT и прочей логики (Sensors/History) из вашего v1.5
// ...
yield(); // Важно для фоновых процессов ESP32
}
```